

Proposta de Domínio — Sistema de Reserva de Serviços

Disciplina: Lab. de Desenvolvimento de Aplicações Móveis e Distribuídas

Curso: Engenharia de Software — PUC Minas

Semestre: 1º Semestre 2026

Aluno: Guilherme Vieira

1. Domínio Escolhido

O sistema desenvolvido neste projeto é uma **plataforma de reserva de serviços**, na qual clientes podem buscar e agendar serviços oferecidos por prestadores cadastrados, e estes prestadores gerenciam as solicitações recebidas em tempo real.

O domínio foi inspirado em plataformas reais como GetNinjas e Thumbtack, adaptado para demonstrar os requisitos arquiteturais da disciplina: comunicação distribuída, arquitetura orientada a eventos e aplicativo móvel.

2. Justificativa

A escolha se justifica por três razões principais:

Aderência aos requisitos arquiteturais. O fluxo natural do domínio — cliente solicita, prestador responde — mapeia diretamente sobre a arquitetura orientada a eventos exigida: a criação de uma reserva é um evento que deve notificar o prestador de forma assíncrona, e a resposta do prestador é um evento que deve notificar o cliente. O MOM (RabbitMQ) aparece como necessidade genuína do negócio, não como adição artificial.

Distinção clara entre perfis. Os dois papéis possuem responsabilidades bem definidas e não se sobrepõem: o cliente consome serviços, o prestador os oferece e gerencia. Isso permite explorar controle de acesso por roles, rotas protegidas e experiências de uso distintas dentro do mesmo aplicativo.

Representatividade do mundo real. Serviços como limpeza, manutenção, tutoria e consultoria são reservados diariamente por aplicativos. O domínio é familiar, o que facilita a validação das funcionalidades e a demonstração do sistema em operação.

3. Perfis de Usuário

3.1 Cliente (**role: CLIENT**)

Usuário que busca e contrata serviços. Suas responsabilidades no sistema são:

- Navegar pelo catálogo de serviços disponíveis
- Visualizar detalhes de um serviço (descrição, preço, duração)
- Criar uma reserva informando data e horário desejados
- Acompanhar o status das próprias reservas em tempo real
- Receber notificações quando o prestador aceitar, recusar ou concluir o serviço

3.2 Prestador (**role: PROVIDER**)

Profissional autônomo ou empresa que oferece serviços pela plataforma. Suas responsabilidades são:

- Cadastrar os serviços que oferece (nome, descrição, preço, duração)
- Receber notificações em tempo real de novas solicitações de reserva
- Aceitar ou recusar reservas pendentes
- Acompanhar as reservas aceitas e marcá-las como concluídas

Um usuário é cadastrado exclusivamente como CLIENT ou PROVIDER, nunca os dois simultaneamente.

4. Principais Funcionalidades

Funcionalidade	Perfil	Descrição
Cadastro e login	Ambos	Autenticação com JWT; role definida no cadastro
Catálogo de serviços	CLIENT	Listagem e busca de serviços disponíveis
Detalhes e reserva	CLIENT	Visualizar serviço e criar reserva com data/hora

Funcionalidade	Perfil	Descrição
Acompanhamento de reservas	CLIENT	Ver histórico e status atual de cada reserva
Notificação de nova reserva	PROVIDER	Receber alerta em tempo real ao ser solicitado
Gestão de solicitações	PROVIDER	Aceitar ou recusar reservas pendentes
Cadastro de serviços	PROVIDER	Criar e editar os serviços que oferece
Conclusão de serviço	PROVIDER	Marcar uma reserva aceita como concluída

5. Fluxo Principal do Sistema

1. Cliente se cadastra com role CLIENT
2. Prestador se cadastra com role PROVIDER e cadastra seus serviços
3. Cliente navega no catálogo e escolhe um serviço
4. Cliente cria uma reserva (data e horário desejados)
5. Backend registra a reserva com status PENDING e publica evento no MOM
6. Prestador recebe notificação em tempo real no app
7. Prestador aceita ou recusa a reserva
8. Backend atualiza o status e publica novo evento no MOM
9. Cliente recebe notificação em tempo real da decisão
10. Se aceita, o prestador realiza o serviço e marca como CONCLUÍDO
11. Cliente recebe notificação final de conclusão

Estados da reserva:

PENDING → ACCEPTED → COMPLETED

→ REFUSED

6. Arquitetura Resumida

O sistema é composto por quatro componentes principais:

- **App Mobile (Flutter):** único aplicativo com experiências distintas por role. Comunica-se com o backend via REST (operações síncronas) e WebSocket (atualizações em tempo real).
 - **Backend REST (Node.js + Express + TypeORM):** expõe endpoints RESTful, aplica regras de negócio, persiste dados no PostgreSQL e publica eventos no RabbitMQ.
 - **MOM (RabbitMQ):** middleware de mensagens responsável pela comunicação assíncrona. Eventos publicados pelo backend são consumidos e roteados via WebSocket para os usuários corretos.
 - **Banco de Dados (PostgreSQL):** persistência central das entidades do domínio. O app mobile utiliza SQLite local para cache offline.
-

7. Tecnologias

Componente	Tecnologia
App Mobile	Flutter / Dart
Backend	Node.js + Express + TypeScript + TypeORM
Banco de dados	PostgreSQL
Cache mobile	SQLite (sqlite)
MOM	RabbitMQ
Autenticação	JWT (JSON Web Token)
Infraestrutura	Docker + Docker Compose